AI-POWERED FACE RECOGNITION SYSTEM FOR MISSING PERSONS

A Synopsis

for

Minor Project

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

(1)Vishesh Mishra

EN23CS3011137

(2)Veer Vachher

EN23CS3011125

(3)Vashistha Rathore

EN23CS3011114

Under the Guidance of

Prof. Kumar Gaurav

Prof. Shilpa Raghuvanshi

MEDICAPS UNIVERSITY

Department of Computer Science & Engineering Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL 2026

Synopsis Contents:

---

1. Introduction

Missing persons represent a critical humanitarian challenge faced by law enforcement agencies, NGOs, and families worldwide. Traditional search methods — distributing photographs, manual identification at checkpoints, and public announcements — are time-consuming, resource-intensive, and prone to human error. With rapid advances in deep learning and computer vision, automated facial recognition systems offer a promising and scalable alternative.

This project proposes an AI-Powered Face Recognition System for Missing Persons, a real-time surveillance application that leverages state-of-the-art deep learning models to automatically identify registered missing individuals from live camera feeds. The system integrates the DeepFace library — specifically the Facenet model — to extract high-dimensional facial embeddings that are robust to variations in lighting, angle, and expression [1].

The system provides a web-based administrative interface built using the Flask framework, allowing operators to register missing persons with a photograph and personal details. A dedicated camera recognition module continuously analyzes webcam frames, compares detected faces against the stored database using cosine similarity, and generates timestamped alerts when a match is found above a defined confidence threshold [2].

By automating the identification process, this system aims to significantly reduce the response time for locating missing individuals, thereby improving outcomes in time-critical situations.

2. Literature Review

The field of automated face recognition has witnessed transformative progress over the past decade, driven by deep learning techniques and large-scale datasets.

Taigman et al. [3] introduced DeepFace, Facebook's deep neural network that approaches human-level performance in facial verification using a 9-layer deep neural network and 3D face alignment. Their work demonstrated that learning face representations from large datasets yields highly discriminative features. Similarly, Schroff et al. [4] proposed FaceNet, which learns a direct mapping from face images to a compact Euclidean space (128 dimensions) where distances directly correspond to face similarity. The FaceNet model, integrated into the DeepFace library, forms the recognition backbone of the proposed system.

Haar Cascade Classifiers, introduced by Viola and Jones [5], provide an efficient real-time face detection mechanism based on Haar-like features and AdaBoost classification. Despite the emergence of deep learning-based detectors, Haar Cascades remain widely used for resource-constrained real-time applications due to their low computational cost.

Several systems for missing person identification using face recognition have been proposed in literature. Kumar et al. [6] developed a cloud-based face recognition system for child trafficking identification using the VGGFace model, achieving 94.3% accuracy. Jain and Learned-Miller [7]

surveyed face recognition systems for law enforcement applications, highlighting the importance of robustness to pose variation and occlusion in real-world deployments.

Flask, a lightweight Python web framework, has been extensively used for rapid prototyping of AI-integrated web applications [8]. MySQL remains the most widely adopted open-source relational database management system for storing structured data in such applications [9].

The present system builds upon these foundational works by combining real-time face detection, deep learning-based face encoding, a relational database backend, and a web interface into a unified, deployable missing person identification platform.

3. Problem Definition

Every year, thousands of individuals are reported missing across India and worldwide. Law enforcement agencies rely heavily on manual photograph-based identification, which has several critical limitations:

- Human operators cannot continuously monitor multiple camera feeds simultaneously.
- Manual comparison of photographs is slow and error-prone, especially under fatigue.
- Traditional systems lack real-time alerting capabilities when a missing person is spotted.
- There is no centralized, searchable database that links facial biometric data with personal records and generates automatic alerts.

Existing commercial surveillance systems are prohibitively expensive for smaller organizations and do not specifically address the missing persons use case. Open-source solutions are often fragmented, lacking integration between the face recognition engine, alert management, and administrative interface.

The proposed system addresses these gaps by providing an integrated, affordable, and automated solution that continuously monitors camera feeds, identifies registered missing persons in real time, and immediately notifies operators through a web-based dashboard — enabling faster, more accurate, and resource-efficient search operations.

4. Objectives

The primary objectives of this project are:

- To develop a real-time face recognition system capable of detecting registered missing persons from live webcam feeds with a confidence threshold-based matching mechanism.
- To implement a deep learning-based facial encoding pipeline using the DeepFace library (Facenet model) producing 128-dimensional embeddings that are robust to lighting, angle, and expression variations.
- To build a web-based administrative portal using Flask for registering missing persons, uploading photographs, and viewing triggered alerts.
- To design and implement a MySQL database schema for persisting person records, facial encodings, and alert logs with referential integrity.
- To implement position-based multi-face tracking enabling simultaneous, independent identification of multiple registered persons in a single camera frame.
- To generate automated, timestamped alert records with snapshot photographs whenever a registered missing person is detected, with a cooldown mechanism to prevent duplicate alerts.

- To ensure the system operates efficiently on standard laptop hardware without requiring specialized GPU infrastructure.

5. Methodology

5.1 System Architecture

The system follows a three-tier architecture comprising the Presentation Layer (Flask web application), the Business Logic Layer (face processing and recognition engine), and the Data Layer (MySQL database). The camera recognition module runs as an independent process, periodically synchronizing with the database to reload newly registered persons.

5.2 Face Registration Pipeline

When an operator registers a new missing person through the web interface, the system performs the following steps:

- The uploaded photograph is saved to the uploads/ directory.
- The FaceProcessor module reads the image using OpenCV and detects the largest face using Haar Cascade.
- The detected face region (with 30px padding) is passed to DeepFace.represent() using the Facenet model with enforce_detection=False to handle partially visible faces.
- The resulting 128-dimensional embedding vector is normalized using L2 normalization.
- The encoding is serialized using Python's pickle module and stored as a binary blob in the missing_persons MySQL table alongside the person's name, age, gender, last seen location, and contact details.

5.3 Real-Time Recognition Pipeline

The camera recognition module implements the following pipeline:

- Frame Capture: OpenCV VideoCapture reads frames continuously from the USB webcam.
- Face Detection: Every frame is passed through Haar Cascade (minNeighbors=8, minSize=80x80) to detect face regions, reducing false positives.
- Recognition Processing: Every 5th frame, each detected face region is cropped (with 20px padding) and passed to FaceProcessor.extract_face_encoding() to obtain its DeepFace embedding.
- Cosine Similarity Matching: The extracted embedding is compared against all stored encodings using cosine similarity. A match is declared if similarity >= 0.70.
- Position-Based Tracking: Each detected face is assigned a stable identity key based on its center position (grid cell), preventing label swapping across frames when multiple faces are present.
- Alert Generation: On a confirmed match, if no alert was generated for that person in the past 30 seconds, a timestamped JPEG snapshot is saved and an alert record is inserted into the alerts table.
- Visualization: Matched faces are marked with a green bounding box and a FOUND label; unmatched faces are marked in red as Unknown.

5.4 Technology Stack

Component	Technology / Library
Programming Language	Python 3.11+
Web Framework	Flask 2.x
Face Recognition	DeepFace (Facenet model) — 128-dim embeddings
Face Detection	OpenCV Haar Cascade Classifier

Component	Technology / Library
Database	MySQL 8.x
DB Connector	mysql-connector-python
Image Processing	OpenCV 4.x, NumPy
Serialization	Python pickle (face encoding storage)
Frontend	HTML5, CSS3, Jinja2 Templates

5.5 Database Design

The system uses two primary tables:

- missing_persons: Stores id, full_name, age, gender, last_seen_location, contact_info, photo_path, face_encoding (BLOB), status, and created_at.
- alerts: Stores id, person_id (FK), camera_location, detected_time, confidence_score, and snapshot_path. The foreign key constraint ensures referential integrity; alerts must be deleted before removing a person record.

5.6 Expected Outcomes

- Accurate identification of registered missing persons in real time with confidence scores displayed.
- Simultaneous, independent recognition of multiple persons in a single camera frame.
- Automated alert generation within 2 seconds of detection, accessible through the web dashboard.
- Minimum 80% recognition accuracy under normal indoor lighting conditions using the Facenet model.

References

[1]. Serengil, S. I., & Ozpinar, A. (2020). "LightFace: A Hybrid Deep Face Recognition Framework." 2020 Innovations in Intelligent Systems and Applications Conference (ASYU), IEEE.

[2]. Bradski, G. (2000). "The OpenCV Library." Dr. Dobb's Journal of Software Tools, 25(11), 120-125.

[3]. Taigman, Y., Yang, M., Ranzato, M., & Wolf, L. (2014). "DeepFace: Closing the Gap to Human-Level Performance in Face Verification." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1701-1708.

[4]. Schroff, F., Kalenichenko, D., & Philbin, J. (2015). "FaceNet: A Unified Embedding for Face Recognition and Clustering." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 815-823.

[5]. Viola, P., & Jones, M. (2001). "Rapid Object Detection using a Boosted Cascade of Simple Features." Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), Vol. 1, pp. 511-518..

[6]. Jain, A. K., & Learned-Miller, E. (2011). "FDDB: A Benchmark for Face Detection in Unconstrained Settings." University of Massachusetts, Amherst, Technical Report UM-CS-2010-009.

[7]. Grinberg, M. (2018). "Flask Web Development: Developing Web Applications with Python." O'Reilly Media, 2nd edition.

[8]. DuBois, P. (2013). "MySQL." Addison-Wesley Professional, 5th edition.